

IDONTKNOWITSNAME

Anonymous Author(s)

Abstract

LLM agent frameworks increasingly expose model invocations through structured programming abstractions, with framework runtimes lowering program-level constructs into concrete prompts. We observe that these programmatically generated prompts are inherently heterogeneous: their components exhibit different sharing scopes, lifetimes, and evolution patterns across invocations. However, existing serving systems treat prompts as opaque text, losing this structure and missing opportunities for more effective KV-cache reuse and management.

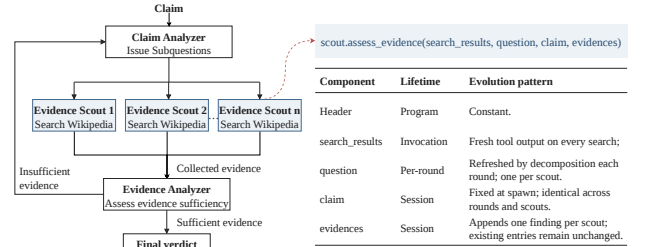
We present IDONTKNOWITSNAME, a compiler-serving co-design that recovers and exploits prompt heterogeneity for efficient KV-cache management. IDONTKNOWITSNAME’s compiler pass performs static analysis and compiler-level optimizations to expose binding lifetimes, expose application-level structure and proactively re-layout prompts to improve cache reuse. The serving runtime exploits the heterogeneity information to optimize KV-cache behavior across requests by prefetching, protection and eviction, according to the predicted future utility of cache entries. We implemented IDONTKNOWITSNAME on SGLang and evaluated it against multiple established serving systems. Our results shows that IDONTKNOWITSNAME.....

1 Introduction

Agent frameworks increasingly expose LLM calls as programming abstractions rather than manual prompt construction. Systems such as ByLLM [2], NVIDIA Object-Oriented Agents (NOOA) [4], and DSPy [6] allow developers to describe model behavior using functions, objects, signatures, and modules, which framework runtimes lower into concrete prompts. This approach also appears in systems such as SIGIL [3], which compiles natural-language skill documents into programs containing typed LLM calls. LLM requests from these frameworks are generated programmatically: they are the serializations of program state rather than free text.

We observed that these programmatically generated LLM requests are inherently heterogeneous. The heterogeneity arises from three dimensions: (1) Sharing scope: one value can be shared across multiple invocations, or only within a single invocation. (2) Lifetime: Different bindings have different lifetimes. Some bindings are single-shot volatile inputs, while others may be reuse across the whole session. (3) Evolution pattern: context for LLM calls can evolve differently overtime, such as remaining fixed, being replaced, or accumulating across invocations. Figure 1 illustrates this heterogeneity in a HoVer-style [5] fact check agent. The claim persists throughout a verification session, the evidence list accumulates over time, and each search result is consumed

once and replaced. Although these values appear in the same model invocation, they offer substantially different reuse opportunities.



```
def assess_evidence(search_results: str, question: str,
                    claim: str, evidences: list[Evidence]) -> Finding by llm()
```

Figure 1. Heterogeneity within a HoVer-style [5] Fact Check Agent’s generated prompt for the summary-making function. `assess_evidence(search_results, question, claim, evidences)` evaluates the sub-question against the accumulated evidence and search results, then answers it.

This distinction matters because main-stream KV-cache serving backbones are prefix-based. In the example, placing the accumulating evidences binding after the short-lived `search_results` binding prevents the model from reusing the evidence prefix whenever the search result changes. The model must therefore recompute KV states for tokens whose contents remain unchanged. Moreover, without knowing which components persist or evolve, the serving system cannot accurately determine which cached tokens should be retained, evicted, or prefetched.

However, existing agent serving systems primarily optimize at the level of requests or agent invocation sequences. Pythia [8] profiles agent workflows to guide cache management, request scheduling, and autoscaling. CacheScout [9] learns execution transitions online to guide eviction and prefetching, while PBKV [10] predicts future invocations and estimates the reuse potential of cached requests. These systems exploit predictability between invocations, but treat the contents of each prompt as an opaque token sequence. Consequently, they cannot identify the heterogeneity within a single prompt, leading to a suboptimal serving performance for such program-integrated agents.

To close the gap, we present IDONTKNOWITSNAME, a compiler-serving co-design that recovers and exploits prompt heterogeneity for efficient KV-cache management. IDONTKNOWITSNAME achieves this by exposing additional prefix reuse and keeping the resulting KV states resident when they are likely to be needed. The identification and exploitation of prompt heterogeneity introduces the following two challenges:

First, heterogeneity is not always explicit in prompt. The sharing scope and lifetime of bindings are closely bonded with the program-level structure and actual runtime assignment. Determining which layout best serves prefix caching is therefore not trivial. IDONTKNOWITSNAME combines static analysis with runtime information to determine prompt layouts. The compiler derives a partial order over prompt components based on their sharing scopes and lifetimes. Then, IDONTKNOWITSNAME uses the actual token lengths of bindings observed from log to order components left unconstrained by this partial order, producing a concrete layout that improves prefix cache efficiency.

Second, exploiting future reuse requires coordinating cache retention and prefetching. Prompt components differ in when they will be reused and how much recomputation their cached prefixes can avoid. Retaining these prefixes consumes capacity needed by other requests, while prefetching too early occupies device memory and prefetching too late fails to hide transfer latency. Inappropriate KV-cache eviction would lead to unnecessary recomputation. IDONTKNOWITSNAME addresses this challenge through two coordinated planners: a protection planner prioritizes cached prefixes according to their expected reuse and benefit, while a prefetch planner schedules transfers based on predicted invocation deadlines and available device memory.

We implemented IDONTKNOWITSNAME on SGLang [11].... We evaluated IDONTKNOWITSNAME with three representative agent workloads: HoVer-style [5] fact check agent, HumanEval [1] code agent, and a BFCL [7] tool-use agent.... Our evaluation shows that IDONTKNOWITSNAME...

In summary, we make the following contributions.

- We identify the inherent heterogeneity in programmatically generated prompts and its implications for serving.
- We design and build IDONTKNOWITSNAME, a compiler-serving co-design that recovers and exploits prompt heterogeneity for efficient KV-cache management....
- We conduct comprehensive evaluations of IDONTKNOWITSNAME against multiple established serving systems, demonstrating that IDONTKNOWITSNAME achieves up to...

2 Background

2.1 Serving systems for LLM agents

2.2 Agent frameworks with LLM calls as programming abstractions

3 Motivation

4 Design

4.1 Overview

In this section, We present the design of IDONTKNOWITSNAME, an efficient serving engine for program-integrated

LLM agents. IDONTKNOWITSNAME adopts a two-tier compiler-server co-design. On the compiler side, IDONTKNOWITSNAME performs static analysis to gather heterogeneity metadata, and on the server side, IDONTKNOWITSNAME leverages this metadata to optimize KV-cache scheduling. We first introduce an interface *Agent Descriptor* to bridge the compiler and server, together with a unified mechanism to construct the agent descriptor via static analysis and runtime observations. Second, we present a heterogeneity-aware KV-cache scheduling strategy that leverages the agent descriptor to enhance serving performance.

4.2 Agent Descriptor for Heterogeneity Metadata

Here we define *Agent Descriptor* to bridge static analysis and server-side scheduling. An agent descriptor contains two parts: (i) **Call-site Template**: It provides a heuristic of the components' lifetime, sharing scope and evolution pattern. (ii) **Workflow Automaton**: It contains the program-level structure of an agent, including workflow dependencies, execution time estimation and transition probabilities. IDONTKNOWITSNAME's two-tier design let the compiler to provide a rough skeleton, while the server refines it with runtime observations. Two sides are therefore connected by this agent descriptor, and jointly serve a more efficient KV-cache scheduling strategy.

4.2.1 Call-site Template. As discussed in §3, the components of a programmatically generated prompt fall into two categories: (i) **Invariant Specifications**: the components that are fixed once the program is compiled, such as the system prompt, response schema, and the tool declarations. They are rendered from the compiler's own intermediate representation and stays constant. (ii) **Dynamic Bindings**: because LLM calls are declared as ordinary functions, the arguments of a call site are supplied in runtime. These runtime values are the dynamic bindings.

We therefore define the call-site template as

$$\mathcal{T}_c = \langle \mathcal{I}_c, \mathcal{B}_c, \mathcal{H}_c \rangle,$$

where $\mathcal{I}_c$ is the invariant specifications, $\mathcal{B}_c$ is the dynamic bindings, and $\mathcal{H}_c$ is the heterogeneity annotation over those bindings. All three are recovered by static analysis. The server derives a layout for $\mathcal{T}_c$ (§4.3.1), under which a valuation of $\mathcal{B}_c$ renders to the served prompt.

Heterogeneity annotation. For each dynamic binding, $\mathcal{H}_c$ records both how its value evolves from one call to the next and how long that value stays valid:

$$\mathcal{H}_c : \mathcal{B}_c \longrightarrow \mathcal{E} \times \mathcal{S}.$$

The evolution component $\mathcal{E}$ classifies a binding by its value provenance. We distinguish $\mathcal{E}$ into five classes, listed from the most to the least reusable:

1. **CONST**: the value is fixed by the program text and is identical at every call.

2. **COPY**: the value is the same as the value another call site carries, so the bytes are already resident once that call site has run.
3. **RESP**: the value comes from the reply produced by another call site.
4. **EXTEND**: the binding is an append-only context, and the previous value remains a prefix of the new one.
5. **VOLATILE**: the value is freshly produced at each call, or its origin lies outside what the analysis can see.

We write ρ for the rank of a class in the order above, which expresses how much of a binding's bytes an earlier call can already supply..

The lifetime component S names the program scope whose completion invalidates a binding's value ($S = \emptyset$ meaning that the value survives for the whole session once produced). For example, `claim` in `scout.assess_evidence` in figure 1 has $\mathcal{E} = \text{CONST}$ and $S = \emptyset$. Lifetime is independent of evolution, and the server needs both. The evolution class governs which bytes a later prompt can reuse, while the lifetime governs when cached bytes can be given up.

4.2.2 Workflow Automaton. An agent typically consists of a sequence of dependent LLM calls whose workflow structure is implicit in the program. Concretely, we abstract an agent as a time-annotated probabilistic finite-state machine (TPFM). We formulate the TPFM as:

$$\mathcal{A} = (Q, P, q_0, F, \mathcal{T}),$$

where:

1. Q is the set of states. Each state $q \in Q$ is labelled by an LLM callsite;
2. P denotes the transition probabilities between states.
3. $q_0 \in Q$ is the entrance callsite.
4. F specifies the termination probability of the agent at each state.
5. $\mathcal{T} = \{D_q^{\text{exec}}, D_{q,q'}^{\text{gap}}\}$ denotes the timing annotations associated with states and transitions, where D_q^{exec} is the execution-time distribution of the LLM callsite represented by state q , and $D_{q,q'}^{\text{gap}}$ is the inter-call gap distribution between the completion of callsite q and the invocation of its successor q' .

Static analysis fixes the shape of this automaton, while its weights comes from runtime logs. The states, the entry, the transitions among them, and final states are recovered from code. $\mathcal{T} = \{D_q^{\text{exec}}, D_{q,q'}^{\text{gap}}\}$ and P are often unknown at compile time and model dependent. The server observes these online and accumulates them across sessions, updating the statistical model of the agent consistently. Together the two halves give the server a forward-looking view of a session.

4.3 Heterogeneity-aware KV-cache Arrangement

4.3.1 Adaptive Prompt re-layout. To best exploit the prefix cache opportunity, we need to arrange the dynamic

bindings in a prompt so that the most reusable ones are placed first.

4.3.2 Proactive KV-cache retention and eviction. 1. Volatile tail will first be evicted and not written to host. (Host Admission policy) 2. Prob*length/time

5 Related Work

6 Conclusion

The inherent structure and heterogeneity of programmatically generated prompts create opportunities for optimization. IDONTKNOWITSNAME identifies and exploits these opportunities rather than treating each prompt as a monolithic unit, thereby unlocking proactive optimizations that improve performance and efficiency. Our evaluation demonstrates that IDONTKNOWITSNAME's methodology successfully creates a more efficient LLM serving system.

References

- [1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).
- [2] Jayanaka L. Dantanarayana, Yiping Kang, Kugesan Sivasothynathan, Christopher Clarke, Baichuan Li, Savini Kashmira, Krisztian Flautner, Lingjia Tang, and Jason Mars. 2025. MTP: A Meaning-Typed Language Abstraction for AI-Integrated Programming. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (2025), 1176–1204. doi:10.1145/3763092
- [3] Jayanaka L. Dantanarayana, Savini Kashmira, Lingjia Tang, and Jason Mars. 2026. SIGIL: Compiling Agent Skills into Typed Harnesses. *arXiv:2607.27309 [cs.SE]* <https://arxiv.org/abs/2607.27309>
- [4] Paul Furgale, Severin Klingler, James Nolan, Matt Staats, Gaia Di Lorenzo, Elisa Martinez Abad, Christian Schüller, Razvan Dinu, Alessio Devoto, Pascal Berard, Gal Kaplun, Elad Sarafian, Riccardo Roveri, Leon Derczynski, and Ricardo Silveira Cabral. 2026. NVIDIA-labs OO Agents: Native Python Object-Oriented Agents. *arXiv:2607.20709 [cs.AI]* <https://arxiv.org/abs/2607.20709>
- [5] Yichen Jiang, Shikha Bordia, Zheng Zhong, Charles Dognin, Maneesh Singh, and Mohit Bansal. 2020. HoVer: A Dataset for Many-Hop Fact Extraction And Claim Verification. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 3441–3460.
- [6] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *International Conference on Learning Representations*.

- [7] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 48371–48392.
- [8] Shan Yu, Junyi Shu, Yuanjiang Ni, Kun Qian, Xue Li, Yang Wang, Jinyuan Zhang, Ziyi Xu, Shuo Yang, Lingjun Zhu, Ennan Zhai, Qingda Lu, Jiarong Xing, Youyou Lu, Xin Jin, Xuanzhe Liu, and Harry Xu. 2026. Pythia: Exploiting Workflow Predictability for Efficient Agent-Native LLM Serving. arXiv:2604.25899 [cs.MA] <https://arxiv.org/abs/2604.25899>
- [9] Rui Zhang, Chaeun Kim, Shaoting Feng, Kuntai Du, Yuhao Liu, Yi Zhong, Cheng-Wei Ching, Junchen Jiang, and Liting Hu. 2026. Learning Agent Execution for KV-Cache Management in Agentic Serving. arXiv:2608.14624 [cs.AI]
- [10] Haoyu Zheng, Fangcheng Fu, Jia Wu, Binhang Yuan, Yongqiang Zhang, Hao Wang, Yuanyuan Zhu, Xiao Yan, and Jiawei Jiang. 2026. Efficient Serving for Dynamic Agent Workflows with Prediction-based KV-Cache Management. arXiv:2605.06472 [cs.LG] <https://arxiv.org/abs/2605.06472>
- [11] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*, Vol. 37.